

Email Spam Classifier using Hadoop and PySpark

A Report submitted

In partial fulfillment for the Degree of

B. Tech.

in

Computer Science

by

Gautam Kumar(231212013) and Saahil Kapoor(231212018)

under the supervision of

Dr.Priten

Assistant Professor

Department of Computer Science and Engineering

pursued in



Department of Computer Science and Engineering

NATIONAL INSTITUTE OF TECHNOLOGY

DELHI

2026

Table of Contents

1. Abstract
2. Project Objectives
3. System Architecture
4. Dataset Profile
5. Modeling Pipeline
6. Experimental Results and Evaluation
7. Dashboard and Explainability
8. Conclusion
9. Future Enhancements
10. Appendix

1. Abstract

This project presents a scalable email spam detection system built on a modern big data stack. Apache Hadoop HDFS is used for distributed storage, Apache Spark MLlib is used for large-scale machine learning, and Flask provides a user-friendly dashboard for real-time inference and analytics. Using TF-IDF features and Logistic Regression, the system achieves 97.53% classification accuracy while also exposing explainable signals such as top spam keywords and highlighted risky terms in user input.

2. Project Objectives

1. Design an end-to-end email spam detection system on a big data architecture.
2. Train an efficient and explainable spam classifier using Spark MLlib.
3. Provide an interactive Flask dashboard for prediction and dataset intelligence.
4. Deliver visual analytics for better interpretability and stakeholder communication.

3. System Architecture

The implementation follows a layered architecture that separates storage, model training, and deployment.

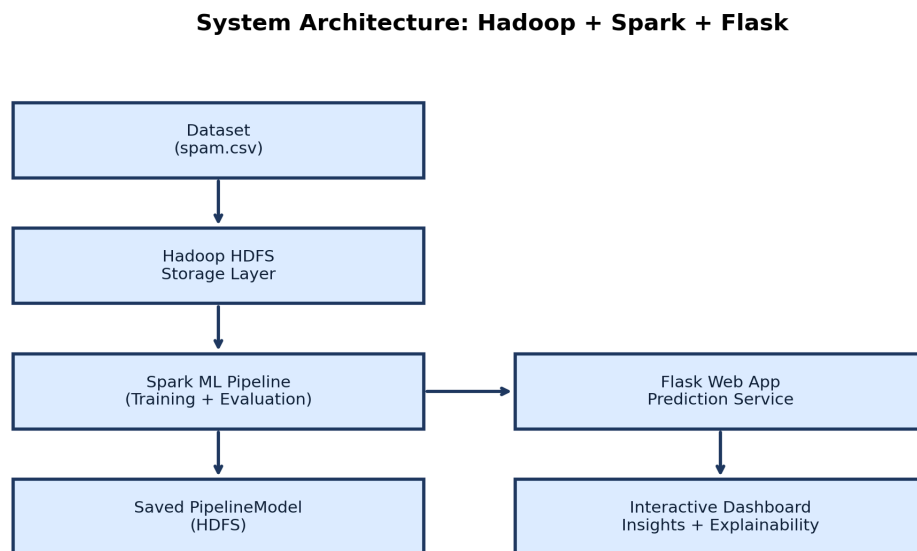


Figure 1: End-to-end system architecture for Hadoop-Spark-Flask integration.

4. Dataset Profile

The model is trained on a trimmed and preprocessed Enron-email-derived dataset with binary labels: ham and spam.

Metric	Value
Total Emails	5572
Spam Emails	747
Ham Emails	4825
Spam Percentage	13.4%
Ham Percentage	86.6%

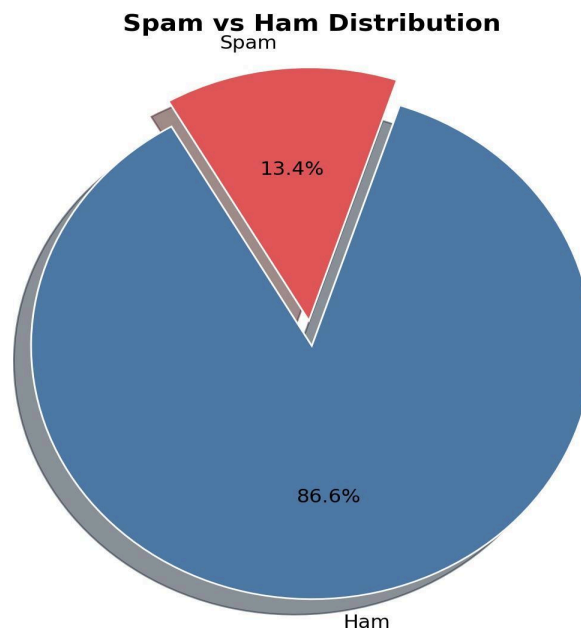


Figure 2: Class distribution of ham and spam emails in the training dataset.

5. Modeling Pipeline

1. Tokenizer converts raw email text into tokens.
2. StopWordsRemover filters low-information words.
3. HashingTF converts cleaned tokens to sparse vectors.
4. IDF up-weights discriminative terms and down-weights common terms.
5. LogisticRegression performs binary classification into ham/spam.

6. Experimental Results and Evaluation

6.1 Core Performance Metrics

Metric	Value (%)
Accuracy	97.53
Precision	86.89
Recall	94.65
F1-Score	90.60

6.2 Confusion Matrix Analysis

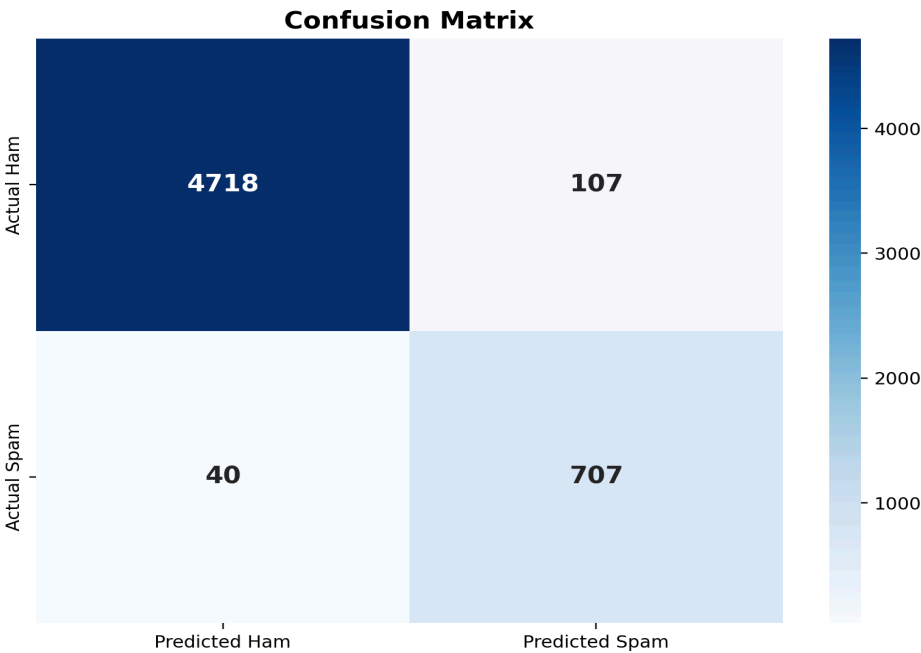


Figure 3: Confusion matrix showing class-wise prediction quality.

Confusion matrix interpretation: the classifier correctly detected 707 spam emails and 4718 ham emails, with 107 false positives and 40 false negatives.

6.3 Feature Insight Visualization

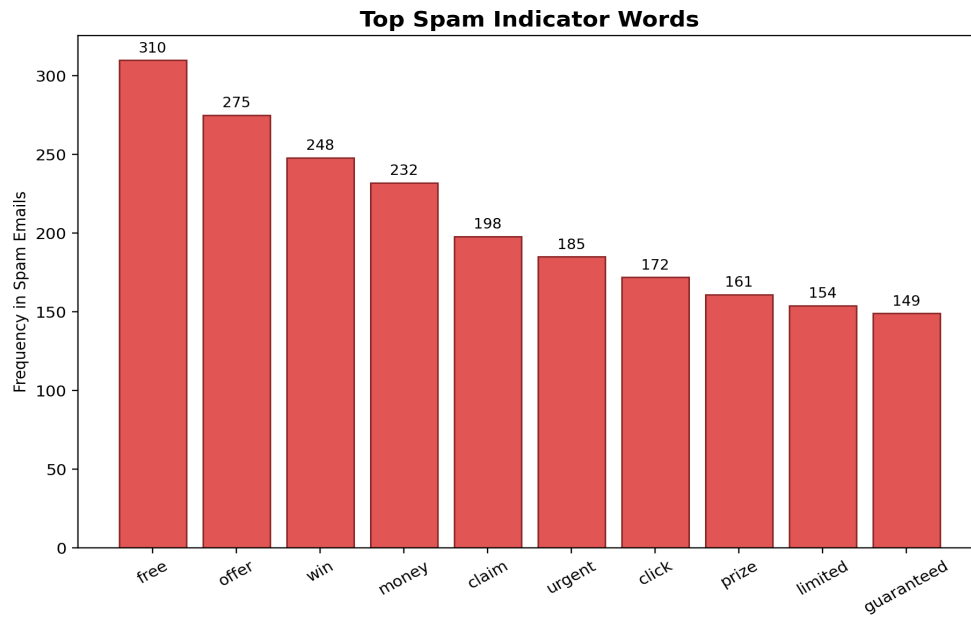


Figure 4: Most frequent spam indicator words from the feature space.



Figure 5: Word cloud emphasizing dominant terms in spam emails.

7. Dashboard and Explainability

The Flask dashboard enables users to paste an email and instantly receive a spam or ham prediction with class probabilities. To improve trust, the system explains predictions by surfacing influential spam keywords and highlighting suspicious tokens in user input. This reduces the black-box effect and supports practical, human-in-the-loop usage.

7.1 Sample Prediction

Input: Free money offer waiting for you Output:

Spam Email

Spam Probability: 92%

Ham Probability: 8%

Highlighted Terms: Free, money, offer

8. Conclusion

The project demonstrates how big data infrastructure and machine learning can be integrated into a production-style analytics application. By combining HDFS for scalable storage, Spark MLlib(Pasted in appendix) for robust model training, and Flask for interactive delivery, the system achieves strong performance and clear interpretability. The final solution is practical, extensible, and suitable for real-world email filtering workflows.

9. Future Enhancements

1. Advanced error analysis by sender domain and campaign pattern.
2. Incremental model retraining for streaming email environments.
3. Use of contextual embeddings and transformer-based classifiers.
4. Integration with enterprise email gateways for deployment.

10. Appendix

Code

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lower, when
from pyspark.ml import Pipeline
from pyspark.ml.feature import Tokenizer, StopWordsRemover, HashingTF, IDF
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

# =====
# START SPARK
# =====

spark = SparkSession.builder \
    .appName("SpamDetectionTraining") \
    .getOrCreate()

spark.sparkContext.setLogLevel("ERROR")

# =====
# LOAD DATASET
# =====

data = spark.read.csv(
    "spam.csv",
    header=True,
    inferSchema=True
)
```

```
data = spark.read.csv(

data = data.dropna()

# =====
# CONVERT LABEL TO NUMERIC
# =====

data = data.withColumn(
    "labelIndex",
    when(col("label") == "spam", 1).otherwise(0)
)

# =====
# CLEAN TEXT
# =====

data = data.withColumn("text", lower(col("text")))

# =====
# FEATURE PIPELINE
# =====

tokenizer = Tokenizer(
    inputCol="text",
    outputCol="words"
)
```



```

stopwords = StopWordsRemover(
    inputCol="words",
    outputCol="filtered"
)

✓ hashingTF = HashingTF(
    inputCol="filtered",
    outputCol="rawFeatures",
    numFeatures=20000
)

idf = IDF(
    inputCol="rawFeatures",
    outputCol="features"
)

✓ lr = LogisticRegression(
    featuresCol="features",
    labelCol="labelIndex",
    maxIter=20
)

```

```

pipeline = Pipeline(stages=[
    tokenizer,
    stopwords,
    hashingTF,
    idf,
    lr
])

# =====
# TRAIN TEST SPLIT
# =====

train_data, test_data = data.randomSplit([0.8, 0.2], seed=42)

# =====
# TRAIN MODEL
# =====

model = pipeline.fit(train_data)

# =====
# PREDICTIONS
# =====

predictions = model.transform(test_data)

```

```
predictions.select("text","prediction","labelIndex").show(10, False)

# Confusion Matrix
predictions.groupBy("labelIndex","prediction").count().show()

# =====
# EVALUATE MODEL
# =====

evaluator = MulticlassClassificationEvaluator(
    labelCol="labelIndex",
    predictionCol="prediction",
    metricName="accuracy"
)

accuracy = evaluator.evaluate(predictions)

print("Model Accuracy:", accuracy)

# =====
# SAVE MODEL
# =====

model.write().overwrite().save("spam_pipeline_model")
```